

Rerunning 2024: A Simulation of Alternative Voting Rules

Lasya Maganti
McKelvey School of Engineering
Washington University in St. Louis
Email: l.maganti@wustl.edu

Abstract—This project investigates how different voting systems would behave if applied to the 2024 U.S. presidential election. Using state-level voter data from seven battleground states, I simulate election outcomes under plurality, Instant Runoff Voting (IRV), Condorcet voting, and approval voting. Because ranked ballots are not available for real elections, I generate synthetic ranked ballots by using the real vote percentages to assign the first choices and then assigning second choices using weighted probabilities based on similar ideologies. Across most simulations, plurality, IRV, and Condorcet elect the same candidate, reflecting the polarized structure of the electorate. Approval voting was much more sensitive than the other systems. When voters were only allowed to approve a small number of candidates, the results looked very similar to plurality. However, when voters could approve more candidates, the winner sometimes shifted toward the candidate who was more broadly acceptable, even if they were not the plurality winner. Overall, this suggests that changing the voting rule does not drastically change the outcome if the electorate is very polarized, except in systems like approval voting that are designed to capture a broader consensus.

I. INTRODUCTION

Election outcomes depend not only on voter preferences but also on the voting system used to analyze those preferences. In the United States, presidential elections use plurality voting, where each voter selects one candidate and the candidate with the most votes wins. While simple, plurality voting ignores information about voters' secondary preferences which can take support away from a more broadly accepted candidate.

Alternative voting systems attempt to address these limitations. Instant Runoff Voting (IRV), Condorcet methods, and approval voting are all voting systems that take in more information.

In this project, I simulate how the 2024 U.S. presidential election might have changed under these alternative systems. I focus on seven competitive states: Arizona, Georgia, Pennsylvania, Michigan, Wisconsin, Nevada, and North Carolina, where small changes in vote aggregation could greatly affect the outcome. The goal of this project is not to predict a true alternative outcome, but to explore how different voting rules respond to the same electorate.

II. BACKGROUND

Different voting systems reflect different ideas about how voter preferences should be assessed, and the differences can lead to varying election outcomes even when the candidate preferences are the same. In plurality voting, only first-choice

votes are counted, which allows a candidate to win without majority support. IRV tries to fix this issue by allowing voters to rank candidates and reallocating votes as candidates are eliminated. This system favors candidates who have strong first-choice support and support from other voters.

Condorcet methods, on the other hand, compare candidates in pairwise elections. A Condorcet winner is defined as a candidate who would defeat every other candidate in a head-to-head majority vote. Condorcet winners are not guaranteed to exist, especially in electorates with cyclical preferences, but they are likely to happen when voter preferences are polarizing.

Approval voting differs from ranking-based systems by allowing voters to approve any number of candidates rather than ranking them. This system allows voters to express broader support and it can favor candidates who are widely acceptable, even if they are not the top choice for most voters. However, the outcome of approval voting depends strongly on how many candidates voters choose to approve, making it sensitive to assumptions about voter behavior.

Because real elections do not provide ranked ballots, simulations must rely on modeling assumptions. In this project, I assume that voters' first choices match the 2024 state-level results, while second-choice preferences depend on ideological similarity between candidates. These assumptions create a polarized but structured electorate, providing a useful and realistic framework for comparing how different voting systems perform.

III. METHODS

A. Data and Candidate Set

I account for six candidates: Trump, Harris, Kennedy, Stein, West, and Oliver. For each of the seven states studied, first-choice votes are taken from 2024 election results or election-night estimates. These values are treated as fixed probabilities from which voters' first choices are drawn.

B. Synthetic Ballot Construction

Because ranked ballots are not available, I generate synthetic ranked ballots using a multi-step process. First, each voter's first choice is sampled according to the state-level votes. Second, the voter's second choice is decided using weighted probabilities based on ideological similarity between candidates. These second-preference weights are structured

but include a small random perturbation to avoid identical electorates.

To ensure that approval voting with larger approval thresholds captures meaningful preference information, I also model third-choice preferences. After the first and second choices are selected, the voter's third choice is found using weighted probabilities based on the second-choice candidate's ideological proximity to the remaining candidates. Candidates ranked below third are placed in random order. This allows higher-ranked preferences to be structured while keeping the overall model simple. This procedure produces ranked ballots for each simulated voter while keeping the first-choice distribution. The entire Python implementation used for ballot generation and the rest of the simulation is included in the Appendix.

C. Voting Rules

Each simulated election is evaluated under plurality, IRV, Condorcet, and approval voting. For approval voting, ranked ballots are converted into approval ballots by approving the top k candidates on each ballot, with $k \in \{1, 2, 3\}$. Approval with $k = 1$ is the same as plurality, while larger values of k allow voters to express more preference.

D. Simulation Procedure

For each state, I simulated elections with both 1,000 voters and 100,000 voters. The smaller simulations illustrate sampling variability, while the larger simulations present more accurate outcomes. The full source code for the larger simulation is included in the appendix.

IV. RESULTS

Using the final simulation setup, 2024 state-level first-choice vote shares and structured, polarized second- and third-preference weights, I ran elections under plurality, IRV, Condorcet, and approval voting across seven competitive states. To understand how sampling variation affects outcomes, I first tested a smaller electorate of 1,000 simulated voters and then increased the electorate to 100,000 voters to obtain stable results.

1) *Small dataset (1,000 voters per state)*: With the smaller simulation size, results exhibited a lot of variability due to sampling noise. In most states, Trump remained the winner under plurality, IRV, and Condorcet voting. However, Wisconsin flipped to Harris under all four systems, including plurality and Condorcet. Several states also showed differences between Condorcet and approval winners. These outcomes indicate that when the electorate is small and noisier, alternative voting systems, especially IRV and approval, can represent votes a lot differently compared to plurality.

2) *Large dataset (100,000 voters per state)*: When the number of simulated voters was increased, results became highly stable. Under plurality, IRV, and Condorcet voting, Trump won in all seven states. The preference structure, which was highly polarized, made Condorcet winners easy to get and matched the plurality outcomes.

Approval voting produced the most variation for the large dataset. When voters were allowed to approve only one

State	Plurality	IRV	Condorcet	Approval ($k = 2$)
Arizona	Trump	Trump	Trump	Trump
Georgia	Trump	Trump	Trump	Harris
Pennsylvania	Trump	Trump	Trump	Harris
Michigan	Trump	Trump	Trump	Harris
Wisconsin	Trump	Trump	Trump	Harris
Nevada	Trump	Trump	Trump	Trump
North Carolina	Trump	Trump	Trump	Trump

TABLE I
WINNERS UNDER PLURALITY, IRV, CONDORCET, AND APPROVAL VOTING WITH $k = 2$ USING 100,000 SIMULATED VOTERS.

State	Approval ($k = 1$)	Approval ($k = 2$)	Approval ($k = 3$)
Arizona	Trump	Trump	Harris
Georgia	Trump	Harris	Harris
Pennsylvania	Trump	Harris	Harris
Michigan	Trump	Harris	Harris
Wisconsin	Trump	Harris	Harris
Nevada	Trump	Trump	Harris
North Carolina	Trump	Trump	Harris

TABLE II
APPROVAL VOTING WINNERS AS THE NUMBER OF APPROVED CANDIDATES INCREASES.

candidate ($k = 1$), approval voting exactly matches plurality. When voters were allowed to approve two candidates ($k = 2$), Harris won in Georgia, Pennsylvania, Michigan, and Wisconsin, while Trump continued to win in Arizona, Nevada, and North Carolina. When voters were allowed to approve three candidates ($k = 3$), Harris won in all seven states.

Arizona, Nevada, and North Carolina only flipped to Harris at $k = 3$, indicating that Harris benefited disproportionately from being ranked third by many voters. These shifts did not appear under plurality, IRV, or Condorcet voting, highlighting the sensitivity of approval voting to how much preference information voters are allowed to express.

V. DISCUSSION

The results suggest that when voter preferences are highly structured and polarized, changing the voting rule does not necessarily lead to different outcomes. Plurality, IRV, and Condorcet voting all selected the same winner in every state. This stability happens because plurality and IRV rely heavily on first-choice support, and the preference structure in the simulation was very one-dimensional, making Condorcet winners easy to obtain.

I initially expected Condorcet winners to be rare, as our class examples often highlighted cycles and nonexistence. However, in this simulation, Condorcet winners appeared consistently. This outcome reflects the fact that preferences were generated using ideological similarity along a single dimension, which makes it much easier to obtain a Condorcet winner. In this simulation, Trump acted as a consensus candidate in pairwise comparisons, even when he did not receive majority support under approval voting.

Approval voting produced the most interesting and varied results. As voters were allowed to approve more candidates, the winner shifted toward Harris, who appeared often as a second or third choice across many ballots. In particular, when the third-ranked candidate was modeled using weighted preferences instead of random assignment, which I originally simulated, Harris got support at $k = 3$. This demonstrates how

approval voting can work positively for broader acceptability. The next step I would take for this portion of the project would be to add some sort of threshold for approved votes, which would add another level of complexity with how important broader accessibility could change the election results.

These findings represent the idea that approval voting uses different information than ranking-based systems, especially when voters are allowed enough flexibility. While plurality, IRV, and Condorcet outcomes were mostly locked in by first-choice votes, approval voting represented more information that was otherwise invisible.

VI. LIMITATIONS

This simulation relies on several simplifying assumptions. Most importantly, second- and third-choice preferences are not observed in real election data and had to be modeled. While the ideological weights used in this project are possible, different assumptions could lead to different outcomes. Accurately representing the political preferences of a large and diverse electorate is very difficult, especially given the many factors that influence voter behavior beyond ideology.

Additionally, the preference structure in this simulation is highly polarized. This makes Condorcet winners much more likely than in electorates with more complex or multidimensional preferences. As a result, the ease with that Condorcet winners happen in this simulation should not be interpreted as a general part of real-world elections.

The simulation results are also highly stable due to the use of real 2024 votes and a large number of simulated voters. When the number of voters was smaller, some states produced different outcomes, like Harris winning under all four systems in Wisconsin. As the sample size increased, randomness averaged out and outcomes became more static.

Finally, while the third-ranked candidate was modeled, lower-ranked preferences were still randomized. The next step of this project would be to model all rankings using weighted preferences, which could further affect approval and Condorcet outcomes.

VII. CONCLUSION

This project examined whether alternative voting systems could have changed the outcome of the 2024 election in several competitive states. Using simulated ballots based on real vote shares and structured preference assumptions, I found that plurality, IRV, and Condorcet voting produced the same outcomes in every state. This suggests that when preferences are polarized and closely aligned with first-choice support, changing the voting rule may not substantially alter election results.

Approval voting, however, had different results. As voters were allowed to approve more candidates, the winner shifted toward the candidate with more acceptability rather than the strongest first-choice support. In several states, Harris only won once third-ranked preferences were included, showing how approval voting can include support that other systems ignore.

Overall, the results highlight that the impact of a voting system depends not only on the rule itself, but also on the structure of voter preferences and how much information voters are allowed to express. Although alternative voting systems often produce the same winner, they can still meaningfully change outcomes in cases where a candidate is widely acceptable but not the top choice for most voters.

REFERENCES

CNN. *2024 Presidential Election Results*. Available at: <https://www.cnn.com/election/2024/results/president>.

APPENDIX

Listing 1. Python source code for the simulation.

```
import random

# List of candidates
candidates = ["Trump", "Harris", "Kennedy", "Stein", "West", "Oliver"]

# First-choice per state
state_poll_shares = {
    "AZ": {
        "Trump": 52.2,
        "Harris": 46.7,
        "Kennedy": 0.0,
        "Stein": 0.5,
        "West": 0.0,
        "Oliver": 0.5
    },
    "GA": {
        "Trump": 50.7,
        "Harris": 48.5,
        "Kennedy": 0.0,
        "Stein": 0.3,
        "West": 0.0,
        "Oliver": 0.4
    },
    "PA": {
        "Trump": 50.4,
        "Harris": 48.6,
        "Kennedy": 0.0,
        "Stein": 0.5,
        "West": 0.0,
        "Oliver": 0.5
    },
    "MI": {
        "Trump": 49.7,
        "Harris": 48.3,
        "Kennedy": 0.5,
        "Stein": 0.8,
        "West": 0.1,
        "Oliver": 0.4
    },
    "WI": {
        "Trump": 49.6,
        "Harris": 48.7,
        "Kennedy": 0.5,
        "Stein": 0.4,
        "West": 0.1,
        "Oliver": 0.3
    },
    "NV": {
        "Trump": 50.6,
        "Harris": 47.5,
```

```

51         "Kennedy": 0.0,
52         "Stein": 0.0,
53         "West": 0.0,
54         "Oliver": 0.4
55     },
56     "NC": {
57         "Trump": 50.9,
58         "Harris": 47.6,
59         "Kennedy": 0.0,
60         "Stein": 0.4,
61         "West": 0.2,
62         "Oliver": 0.4
63     }
64 }
65
66 # weights for second choice based on
67   similarity of ideals
68 second_pref_weights = {
69     "Trump": {
70         "Harris": 0.10,
71         "Kennedy": 0.30,
72         "Stein": 0.10,
73         "West": 0.10,
74         "Oliver": 0.40
75     },
76     "Harris": {
77         "Trump": 0.05,
78         "Kennedy": 0.20,
79         "Stein": 0.40,
80         "West": 0.30,
81         "Oliver": 0.05
82     },
83     "Kennedy": {
84         "Trump": 0.30,
85         "Harris": 0.30,
86         "Stein": 0.10,
87         "West": 0.10,
88         "Oliver": 0.20
89     },
90     "West": {
91         "Trump": 0.05,
92         "Harris": 0.30,
93         "Kennedy": 0.20,
94         "Stein": 0.40,
95         "Oliver": 0.05
96     },
97     "Stein": {
98         "Trump": 0.05,
99         "Harris": 0.30,
100        "Kennedy": 0.20,
101        "West": 0.40,
102        "Oliver": 0.05
103    },
104    "Oliver": {
105        "Trump": 0.40,
106        "Harris": 0.15,
107        "Kennedy": 0.30,
108        "Stein": 0.075,
109        "West": 0.075
110    }
111 }
112
113 # Number of voters to simulate
114 NUM_VOTERS = 100000

```

```

119 # GENERATE RANKED BALLOTS
120
121
122 def choose_first_choice(poll_shares,
123     candidates):
124     """
125     Choose a first-choice candidate using the
126     poll percentages.
127     """
128     total = sum(poll_shares[c] for c in
129         candidates)
130     r = random.random() * total
131     running = 0.0
132     for c in candidates:
133         running += poll_shares[c]
134         if r <= running:
135             return c
136     return candidates[-1]
137
138
139 def weighted_choice(options, weights):
140     """
141     Choose one option from a list, with
142     probabilities proportional to weights.
143     """
144     total = sum(weights)
145     if total <= 0:
146         return random.choice(options)
147
148     r = random.random() * total
149     running = 0.0
150     for i in range(len(options)):
151         running += weights[i]
152         if r <= running:
153             return options[i]
154     return options[-1]
155
156
157 def make_one_ballot(candidates, poll_shares,
158     second_pref_weights):
159     """
160     Pick first choice using poll_shares, pick
161     second and third choice using
162     second_pref_weights based on first
163     choice.
164     """
165     first = choose_first_choice(poll_shares,
166         candidates)
167
168     # second choices (everyone except first)
169     others = [c for c in candidates if c !=
170         first]
171
172     weights_row = second_pref_weights.get(first,
173         {})
174
175     weights = []
176     for c in others:
177         base = weights_row.get(c, 0.1)
178         # small random jitter so different
179         # electorates are slightly different
180         jitter = random.uniform(-0.05, 0.05)
181         w = max(0.01, base + jitter)
182         weights.append(w)
183
184     # choose second choice with weights
185     second = weighted_choice(others, weights)

```

```

176 # remaining candidates after choosing
177     second
178 remaining = [c for c in others if c !=
179             second]
180
181 # use second choice weights to do third
182     choice
183 third_weights_row = second_pref_weights.get(
184     second, {})
185
186 third_weights = []
187 for c in remaining:
188     base = third_weights_row.get(c, 0.1)
189     jitter = random.uniform(-0.05, 0.05)
190     w = max(0.01, base + jitter)
191     third_weights.append(w)
192
193 third = weighted_choice(remaining,
194     third_weights)
195
196 rest = [c for c in remaining if c != third]
197 random.shuffle(rest)
198 ballot = [first, second, third] + rest
199 return ballot
200
201 def generate_ballots(num_voters, candidates,
202     poll_shares, second_pref_weights):
203     """
204     Make ranked ballots.
205     """
206     ballots = []
207     for i in range(num_voters):
208         ballots.append(make_one_ballot(
209             candidates, poll_shares,
210             second_pref_weights))
211     return ballots
212
213 # PLURALITY VOTING
214
215 def plurality_winner(ballots, candidates):
216     counts = {}
217     for c in candidates:
218         counts[c] = 0
219
220     for ballot in ballots:
221         first = ballot[0]
222         counts[first] += 1
223
224     winner = None
225     max_votes = -1
226     for c in candidates:
227         if counts[c] > max_votes:
228             max_votes = counts[c]
229             winner = c
230
231     return winner, counts
232
233 # IRV
234
235 def irv_winner(ballots, candidates):
236     remaining = candidates[:] # copy so remove
237     # eliminated candidate without affecting
238     # the original list
239
240     while True:
241         counts = {}
242         for c in remaining:
243             counts[c] = 0
244
245         total_active = 0
246
247         for ballot in ballots:
248             # find first candidate on this ballot
249             # who is still remaining
250             for c in ballot:
251                 if c in remaining:
252                     counts[c] += 1
253                     total_active += 1
254                     break
255
256         if len(remaining) == 0:
257             return None
258
259         # check for majority
260         for c in remaining:
261             if counts[c] > total_active / 2.0:
262                 return c
263
264         # eliminate lowest candidate
265         min_votes = min(counts[c] for c in
266             remaining)
267         lowest = [c for c in remaining if counts
268             [c] == min_votes]
269
270         elim = random.choice(lowest)
271         remaining.remove(elim)
272
273         if len(remaining) == 1:
274             return remaining[0]
275
276 # CONDORCET WINNER
277
278 def prefers_over(ballot, a, b):
279     # include try/except in case candidate is
280     # not in ballot
281     try:
282         pos_a = ballot.index(a)
283     except ValueError:
284         pos_a = 999
285     try:
286         pos_b = ballot.index(b)
287     except ValueError:
288         pos_b = 999
289     return pos_a < pos_b
290
291 def condorcet_winner(ballots, candidates):
292     num_voters = len(ballots)
293
294     for a in candidates:
295         wins_all = True
296         for b in candidates:
297             if a == b:
298                 continue
299
300             count_a_over_b = 0
301             for ballot in ballots:
302                 if prefers_over(ballot, a, b):
303                     count_a_over_b += 1
304
305             if count_a_over_b <= num_voters /
306                 2.0: # checks if more than half
307                     # the voters prefer a over b
308                     wins_all = False
309                     break

```

```

298         if wins_all:
299             return a
300
301     return None
302
303 # APPROVAL VOTING
304
305 def ranked_to_approval(ballot, k):
306     return ballot[:k]
307
308 def approval_winner_from_ranked(ballots,
309     candidates, k):
310     counts = {}
311     for c in candidates:
312         counts[c] = 0
313
314     for ballot in ballots:
315         approved = ranked_to_approval(ballot, k)
316         for c in approved:
317             counts[c] += 1
318
319     max_approvals = max(counts.values())
320     tied = [c for c in candidates if counts[c]
321         == max_approvals]
322
323     # break ties randomly
324     winner = random.choice(tied)
325
326     return winner, counts
327
328 # RUN
329
330 if __name__ == "__main__":
331     #random.seed(1)
332
333     for state, poll_shares in state_poll_shares
334         .items():
335         print("-----")
336         print("State:", state)
337         print("-----")
338
339         # Generate ballots
340         ballots = generate_ballots(NUM_VOTERS,
341             candidates, poll_shares,
342             second_pref_weights)
343
344         # Plurality
345         pl_w, pl_counts = plurality_winner(
346             ballots, candidates)
347         print("Plurality winner:", pl_w)
348         print("Plurality counts:", pl_counts)
349         print()
350
351         # IRV
352         irv_w = irv_winner(ballots, candidates)
353         print("IRV winner:", irv_w)
354         print()
355
356         # Condorcet
357         cond_w = condorcet_winner(ballots,
358             candidates)
359         print("Condorcet winner:", cond_w)
360         print()

```

```

360     # Approval voting for k = 1, 2, 3
361     for k in [1, 2, 3]:
362         appr_w, appr_counts =
363             approval_winner_from_ranked(
364                 ballots, candidates, k=k)
365         print(f"Approval winner (top {k}
366             approved): {appr_w}")
367         print("\n")

```